

Safety-Gated Digital Twin Architecture for Smart-Elevator Supervision: A Software-Validated Laboratory Prototype

Abderrahmane Zaouidi, Mohamed Hachem Bengherbia, and Mounir Bouhedda

Abstract—Connected supervision makes elevators easier to monitor, but it turns brokers, dashboards, workflow engines, and twin APIs into part of the safety boundary, where the decisive risk is command authority: who or what may move a safety-critical machine. We present an architecture and software-validation study of a laboratory-scale smart-elevator digital twin that addresses this risk through a safety-gated command path. Eclipse Ditto holds the authoritative state; every operator or workflow action becomes command *intent* that a deterministic rule-based gate must admit before it is persisted for audit and forwarded to the device, which still applies firmware interlocks; a rejected command is logged yet produces zero twin writes. The self-hosted platform couples a four-floor ESP32-S3 prototype and an authenticated simulator with MQTT/TLS, a bridge, PostgreSQL/TimescaleDB, bounded n8n workflows, an adaptive dispatch engine, and a SCADA dashboard; workflows and an optional language model explain and recommend but never hold command authority, and the machine-learning dispatch challenger remains a non-authoritative shadow. Validation is reported at the software and integration levels: 127 automated assertions pass; broker authentication, access-control lists, and TLS are exercised; and a 57-command live campaign yields 28 accepted and 29 rejected commands, the reject-with-zero-write invariant holding on all 29 rejections, database-verified. The gate admits a command in a median of 5.7 μ s, about 5.3 μ s over a direct publish, so the safety discipline imposes negligible computational cost on command logic. Floor position is tracked by calibrated open-loop step counting; closed-loop sensing, instrumented latency, and embedded-resource profiling remain future work.

Index Terms—Digital twin, smart elevator, Industrial Internet of Things, Eclipse Ditto, MQTT, SCADA, bounded automation, runtime assurance, safety gate.

I. INTRODUCTION

Elevators are safety-oriented cyber-physical systems whose availability affects mobility, emergency access, maintenance cost, and building security. Connected supervision can provide earlier visibility into overload, vibration, overheating, door faults, unauthorized access, and degraded components. However, connectivity also increases the attack surface: brokers, workflow engines, dashboards,

digital-twin APIs, and configuration endpoints become part of the operational safety boundary.

Many laboratory IoT systems present telemetry directly to a dashboard or allow a web application to publish device commands. That pattern is inappropriate for a safety-oriented elevator platform. In this work, the dashboard never controls the device directly. MQTT transports device telemetry and device-facing commands, but Eclipse Ditto is the operational source of truth. Commands from operators or workflows become Ditto intent only after deterministic safety validation, database logging, and audit recording. The bridge then forwards accepted intent to the device over authenticated MQTT, and the firmware or simulator applies local interlocks before actuation.

The article is framed as an architecture, prototype, and software-and-integration validation study, not as an optimization benchmark or a claim of exhaustive system-level experimental proof. The prototype is a rigorous laboratory platform, not a certified industrial elevator controller. We adopt the evidence boundary of the thesis validation package and report results at three explicit levels: software validation (logic and integration proven by automated tests, exports, and live-stack traces), integration validation (end-to-end behavior of the assembled stack observed on live infrastructure), and documented hardware integration (the built prototype and its observed behavior). Physical mechanisms without captured serial, video, or log measurements are reported as documented integration or future work, and evidence at one level is never taken to imply another.

The paper makes three main contributions:

- a *reproducible, self-hosted supervision architecture* for a smart-elevator digital twin, integrating an ESP32-S3 prototype, Mosquitto, Eclipse Ditto, PostgreSQL/TimescaleDB, bounded n8n workflows, an adaptive dispatch engine, and a Next.js SCADA dashboard;
- a *command-authority discipline* in which Ditto is the state authority, MQTT is the transport layer, and every command follows a deterministic safety-gated intent path with an auditable reject-with-zero-write contract; the workflows and an optional language model explain and recommend but never hold safety authority, and the machine-learning dispatch challenger (Brain B) remains a non-authoritative shadow pending future evidence and human approval;

The authors are with the Laboratory of Advanced Electronic Systems (LSEA), University of Médéa, Médéa, Algeria (e-mails: abdelrahmenzaouidi@gmail.com; mohamed.bengherbia26@gmail.com; bouhedda.mounir@univ-medea.dz).

Corresponding author: Abderrahmane Zaouidi.

- a *quantitative, evidence-honest validation* showing that this safety discipline adds negligible in-process computation (median 5.7 μ s gate decision; about 5.3 μ s over a direct publish), while explicitly separating software validation, integration validation, and documented hardware integration.

The rest of the paper is organized as follows. Section II reviews related work and the research gap. Section III presents the system architecture and command path. Section IV describes the implementation. Section V reports validation evidence and the measurements deferred to future work. Sections VI and VII discuss scope, limitations, and future hardening, and Section VIII concludes.

II. RELATED WORK AND RESEARCH GAP

A. Digital Twins for Cyber-Physical Systems

Digital twins provide persistent virtual representations of physical assets synchronized with operational data and context. The concept has evolved from early virtual-counterpart and aerospace paradigms into industrial reference architectures and surveys of enabling technologies [1]–[4]. Kritzing et al. distinguish digital models, digital shadows, and digital twins according to the degree of automated bidirectional data flow [5]. IoT-oriented reviews identify recurring limitations: heterogeneous models, fragmented tooling, weak reproducibility, and insufficient integration of security and control feedback paths [6], [7].

Eclipse Ditto is useful in this context because it models each asset as a Thing with attributes, features, and policies, consistent with web-of-things concepts for addressable properties and actions [8], [9]. This supports an explicit separation between slowly varying metadata and subsystem state. In the present platform, Ditto is not a dashboard cache. It is the operational state authority through which dashboards, agents, dispatch logic, and command intent are coordinated.

B. Elevator IoT and Predictive Maintenance

Elevator monitoring commonly emphasizes condition indicators such as door cycles, vibration, load, temperature, power, availability, and incident frequency. Predictive-maintenance surveys show the shift from scheduled maintenance to condition-based and predictive strategies, often using digital twins or machine-learning models [10], [11]. These approaches are valuable, but laboratory prototypes rarely have long-duration labeled failure data. The present work therefore implements predictive maintenance as deterministic decision support: risk flags, work-order state, and recommended actions are generated from auditable rules rather than from an unvalidated failure-probability model.

C. Messaging, Security, and Multi-Agent Automation

MQTT is a lightweight publish/subscribe protocol suitable for constrained devices and asynchronous IoT systems [12]–[14]. Mosquitto provides a practical open-source broker

implementation [15]. The protocol alone does not provide a complete industrial security posture, so authenticated identities, topic ACLs, TLS, credential lifecycle management, monitoring, and segmentation are required in safety-relevant deployments [16]–[19].

Multi-agent systems distribute responsibility among cooperating entities [20], [21]. Recent language-model agent literature expands this vocabulary, but a safety-oriented elevator platform cannot allow opaque model outputs to authorize physical commands [22], [23]. We therefore adopt *bounded autonomy* as the primary framing: rather than open-ended agency, the platform uses bounded workflow automation with explicit roles and deterministic authority boundaries. Under this framing the optional local language model produces human-readable explanation only, is advisory without command authority, and is not part of the command-admission predicate.

D. Runtime Assurance and Safety Monitors

The closest prior art to this paper's contribution is not digital-twin modeling but runtime assurance and safety monitoring for cyber-physical and autonomous systems. The Simplex architecture established the pattern of a small, high-assurance decision module that constrains a complex, less-trusted controller so the system stays within a safety envelope [24]. Runtime verification monitors an executing system against formally specified properties and reacts when they are violated [25]. For learning-enabled control, shielding synthesizes a reactive safety layer that filters or corrects actions before they reach the plant, so an unverified policy can never drive the system into an unsafe state [26]. These mechanisms share a lineage with supervisory control of discrete-event systems, where a supervisor restricts a plant to an admissible behavior [27].

The safety-gated command path in this work is an engineering instance of the same principle applied to a connected elevator: the deterministic gate is the high-assurance module; the dashboard, workflows, and optional language model are the complex, untrusted components; and the gate admits a command only if it satisfies the safety predicate of Eq. (1). What is distinctive is the composition with a twin-as-state-authority design and an auditable reject-with-zero-write contract—rejected commands are recorded for audit yet mutate no state—together with bounded workflow agents whose recommendations never carry safety authority. To our knowledge this specific combination of runtime-assurance-style admission, an authoritative digital twin, and evidence-honest validation has not been reported for elevator supervision.

E. Research Gap

Prior work is strong in individual layers: digital-twin modeling, MQTT telemetry, predictive maintenance, IoT security, or automation workflows. The gap addressed here is their disciplined integration into a single reproducible cyber-physical architecture where (i) the twin is authoritative, (ii) command authority is explicit and

auditable, (iii) safety-critical decisions remain deterministic, (iv) MQTT identities and ACLs enforce separation, and (v) validation artifacts distinguish software evidence from physical-hardware claims. Table I contrasts the main prior-art categories with this work across these five dimensions: each line is strong in its own column, but none combines all five, which is the integration this paper targets.

The pattern in Table I is that each prior-art category is strong along one or two dimensions and silent on the rest, and in particular no category combines an authoritative twin (i) with explicit, auditable command authority (ii). The contribution of this paper is therefore not a new mechanism in any single column but the disciplined composition of all five, validated with an explicit evidence boundary.

III. SYSTEM ARCHITECTURE

A. Layered Stack

The implemented stack is organized as the layered architecture of Fig. 1: telemetry flows up from the field to the state authority and its consumers, while every command flows down only through the deterministic safety gate.

The physical layer is a reduced-scale four-floor, single-cabin prototype controlled by ESP32-S3 firmware. The communication layer uses Mosquitto with JSON topics `elevator/{id}/telemetry`, `elevator/{id}/events`, `elevator/{id}/status`, and `elevator/{id}/commands`. The twin layer is Eclipse Ditto, accessed through `/api/2/things/{thingId}`. The history layer is PostgreSQL/TimescaleDB. The agent layer consists of n8n workflows and a dispatch engine. The presentation layer is the Next.js SCADA dashboard.

Table II records the responsibility boundary that guided the implementation. This is a core design decision because failures in cyber-physical systems often arise when a layer quietly assumes authority that belongs elsewhere. The dashboard can request, but not execute. The bridge can forward accepted intent, but not decide safety. The workflow layer can analyze and route, but not override deterministic admission. The firmware remains responsible for local actuation checks even after backend admission.

B. Command Authority Principle

Fig. 2 is the architectural centerpiece. The dashboard and n8n workflows may request commands. The deterministic gate decides whether they can become Ditto intent. The bridge forwards only accepted, fresh, non-duplicate command intents. The firmware or simulator remains the last software boundary before actuation. This design prevents three unsafe shortcuts: browser-to-MQTT command publication, LLM-generated command authorization, and direct workflow mutation of actuator state.

C. Automation Boundary

The n8n layer is decomposed into specialized workflows. The surveillance workflow canonicalizes twin events and

records timelines. The analysis workflow computes deterministic risk flags and optional human-readable explanation. The control workflow applies the safety gate. Security and maintenance workflows update security state and work orders. Notification workflows write and deliver outbox records. Optimization and audit workflows record dispatch decisions and operational traces. This is bounded, multi-workflow automation, and the authority boundary is conservative: risk classification, command admission, and dispatch-policy selection are deterministic and auditable.

Table III summarizes the workflow roles. The important point is not the number of workflows, but the fact that each workflow has a bounded contract and produces explicit state or audit artifacts. In particular, the analysis workflow may attach an LLM-generated explanation when the local model is enabled, but the command gate does not read that explanation as an authorization signal.

IV. IMPLEMENTATION

A. Prototype and Firmware

Table IV summarizes the physical prototype, shown in Fig. 3. The ESP32-S3 was selected for Wi-Fi connectivity, GPIO, ADC, SPI/I2C, timers, PWM-capable pins, and sufficient memory for MQTT/TLS operation in a laboratory prototype [28], [29].

Two aspects of Table IV should be retained. First, the backend stack is exercised against a real electromechanical plant with real actuation and timing, not only against a software model: the prototype and the simulator implement the same MQTT contract, so integration evidence gathered with either endpoint applies to the same interface. Second, the table itself encodes the evidence boundary of contribution three—floor position is tracked functionally in open loop, and safety telemetry is potentiometer-simulated—so the reader can see, per subsystem, which claims are physical and which are not.

The deployed firmware implements a finite-state machine with idle, door-opening, door-wait, door-closing, moving-up, moving-down, arrival, error-stop, and emergency states. Remote commands are not direct GPIO operations; they are parsed into state-machine requests and checked against firmware-side interlocks.

The firmware publishes JSON telemetry to `elevator/building-floor1-elevator/telemetry`, events to `elevator/building-floor1-elevator/events`, retained status to `elevator/building-floor1-elevator/status`, and subscribes to `elevator/building-floor1-elevator/commands`. Firmware telemetry uses a Ditto-aligned feature envelope, and the repository configuration sets a 3000 ms periodic telemetry interval when not moving, a 5000 ms status interval, and position-oriented updates during motion. These are configured design parameters unless instrumented captures are added.

B. MQTT Security Baseline

The broker disables anonymous access and uses per-client identities: `esp32-elevator`, `bridge`, `agents`, `dashboard`,

TABLE I

PRIOR-ART CATEGORIES VERSUS THIS WORK ACROSS THE FIVE GAP DIMENSIONS. Y: ADDRESSED; P: PARTIAL; -: NOT ADDRESSED. DIMENSIONS: (I) TWIN AS STATE AUTHORITY; (II) EXPLICIT, AUDITABLE COMMAND AUTHORITY; (III) DETERMINISTIC SAFETY-CRITICAL DECISIONS; (IV) MQTT IDENTITY/ACL SEPARATION; (V) EVIDENCE-HONEST VALIDATION.

Approach (representative work)	(i)	(ii)	(iii)	(iv)	(v)	Primary focus
Digital-twin modeling [3], [5], [6]	P	–	–	–	–	Asset representation and synchronization
Elevator IoT / predictive maintenance [10], [11]	P	–	–	–	–	Condition monitoring and failure prediction
Runtime assurance / shielding / supervisory control [24], [26], [27]	–	P	Y	–	P	Keeping an untrusted controller within a safety envelope
IoT security baselines [17], [19]	–	–	–	Y	–	Authentication, ACLs, and segmentation
LLM / multi-agent automation [22], [23]	–	P	–	–	–	Flexible task automation
This work	Y	Y	Y	Y	Y	Safety-gated command authority on an authoritative twin

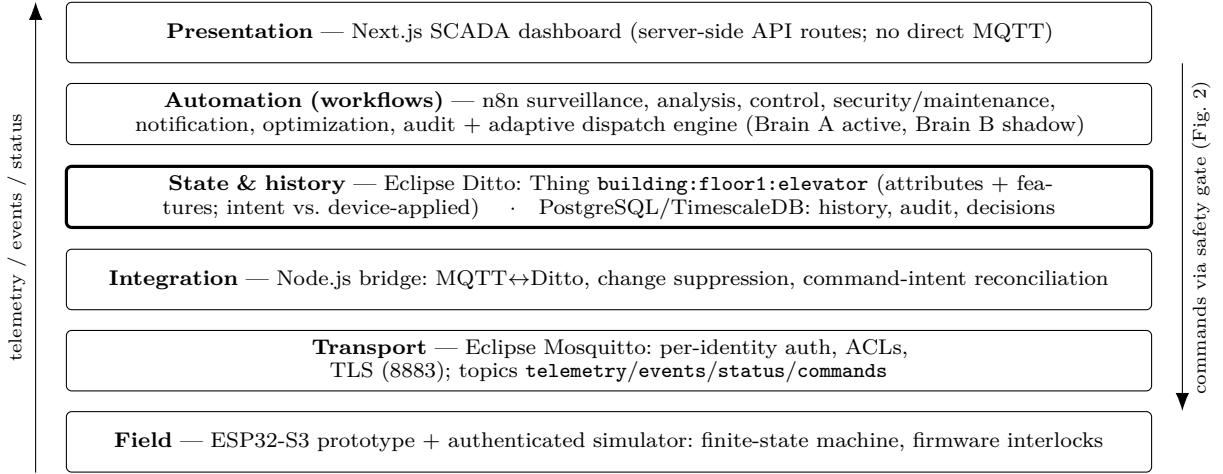


Fig. 1. Layered architecture. Telemetry flows up from the field to the state authority and its consumers; every command flows down only through the deterministic safety gate of Fig. 2. Eclipse Ditto, not the dashboard, is the operational source of truth.

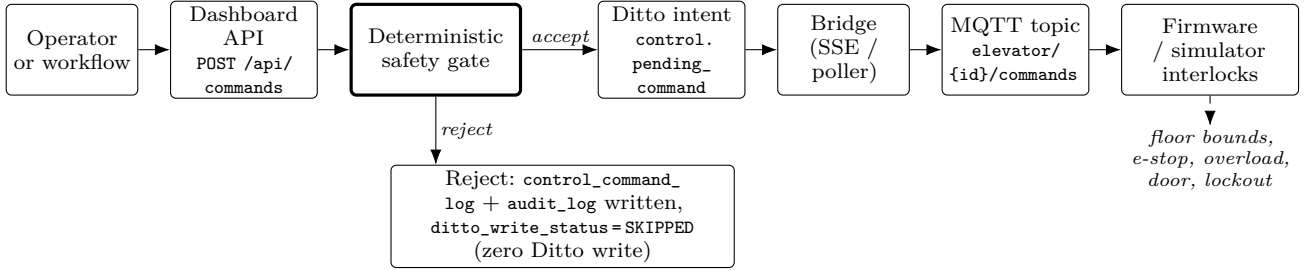


Fig. 2. Safety-gated command path. The browser, workflows, and optional LLM never publish device commands directly. Accepted intent is persisted as Ditto command intent and forwarded by the bridge; rejected commands are logged for audit but produce zero Ditto writes (`ditto_write_status = SKIPPED`). The firmware or simulator applies local interlocks before actuation.

and **healthcheck**. The ESP32 identity publishes only its telemetry, events, and status and subscribes only to its commands. The dashboard identity is read-only for diagnostics and cannot publish command topics. The bridge identity can publish device commands after Ditto intent is accepted. The security evidence package shows anonymous and wrong-password connections rejected with **not authorised**, while an authenticated bridge subscriber received telemetry. A separate field TLS evidence capture on port 8883 confirms successful publish/subscribe over the ESP32-side TLS listener; the steady-state TLS round-trip time was measured separately on a loopback broker

(Section V).

The topic convention is deliberately narrow. Periodic telemetry carries feature-shaped JSON snapshots; events carry discrete safety, security, and maintenance incidents; status carries retained presence; and commands are cloud-to-device messages produced only by authorized back-end identities. This separation allows ACLs to express operational authority: an ESP32 credential can never publish commands, and a dashboard credential can never bypass the server-side command API. In a multi-building deployment, the same convention extends through Thing IDs and MQTT-safe identifiers without changing the UI

TABLE II
RESPONSIBILITY BOUNDARIES IN THE IMPLEMENTED ARCHITECTURE

Layer	Owns	Must not own	Reliability/security rationale
ESP32-S3 firmware	Local state machine, actuator control, emergency and movement interlocks, MQTT telemetry and command subscription.	Fleet history, operator identity, long-term analytics, or direct dashboard state.	Keeps immediate protection close to the physical process and allows safe local behavior during network loss.
MQTT broker	Authenticated message transport, topic routing, retained status, ACL enforcement.	Operational truth, UI state, or command authorization.	Avoids treating transient wire messages as the authoritative model.
Bridge	Telemetry normalization, Ditto feature writes, command-intent reconciliation, MQTT command forwarding.	Human authorization, risk acceptance, or hidden business logic.	Makes the bridge auditable and recoverable after restart; accepted intent can be reconciled from Ditto.
Eclipse Ditto	Current operational state, attributes, subsystem features, command intent, policy boundary.	Long-term history or direct motor execution.	Gives all consumers one source of truth while preserving device-side interlocks.
PostgreSQL/TimescaleDB	Telemetry history, audit, command decisions, notifications, maintenance and dispatch records.	Current operational authority.	Enables traceability and analytics without making stale database rows control the elevator.
n8n workflows	Surveillance, analysis, maintenance, security, notifications, audit routing, bounded command requests.	Safety-critical command admission or LLM-based authorization.	Enables bounded automation while keeping decisions inspectable and testable.
Next.js dashboard	SCADA Operator visualization, history views, command-request UI, server-side API calls.	Direct MQTT publication or client-side secrets.	Prevents browser bypass of Ditto, safety gate, and audit logging.

TABLE III
BOUNDED WORKFLOW AUTOMATION DECOMPOSITION

Workflow role	Trigger	Decision/action	Evidence or state written
Surveillance workflow	Ditto change, MQTT-derived twin event, scheduled scan.	Canonicalize event, deduplicate, update timeline, preserve correlation identifiers.	Telemetry/history rows and normalized event envelope.
Analysis workflow	New telemetry/risk event.	Apply deterministic risk rules; optionally request explanatory text from local LLM.	<code>ai_analysis</code> feature, risk flags, audit rows.
Control workflow	Operator or workflow command request.	Run deterministic command safety gate and branch accepted/rejected paths.	<code>control_command_log</code> , <code>audit_log</code> , Ditto writes only if accepted.
Security workflow	RFID/security/forced-door event or risk escalation.	Update security state, request lockdown only through the gate, surface human-review requirement.	<code>security</code> feature and audit rows.
Maintenance workflow	Wear/risk threshold, repeated incident, scheduled review.	Create or update work-order state using rule-based evidence.	<code>maintenance_schedule</code> , <code>maintenance_work_orders</code> .
Notification workflow	Alert or outbox due event.	Build dedupe key, select channels, retry or mark failed/sent.	<code>notification_outbox</code> .
Optimization/audit workflow	Dispatch cycle or audit event.	Record dispatch decisions, policy changes, reward/proxy outcomes, workflow activity.	Dispatch/audit tables and dashboard history APIs.

control model.

C. MQTT-to-Ditto Bridge

The Node.js bridge subscribes to `elevator/+/telemetry`, `elevator/+/events`, and `elevator/+/status`, parses JSON, resolves the Thing ID, maps aliases to canonical field names, groups values into Ditto features, suppresses unchanged writes, retries Ditto writes, and updates microcontroller presence. It writes specific feature paths under `/api/2/things/{thingId}` rather than replacing the full Thing document.

The reverse path is implemented through Ditto Server-Sent Events plus a polling recovery path for `features/control/properties/pending_command`. The bridge filters stale, duplicate, rejected, or already-forwarded commands. Accepted intents are translated to compact MQTT commands such as `MOVE_TO_FLOOR`, `EMERGENCY_STOP`, `RESET`, and `DISPATCH_POLICY`.

D. Ditto Thing Model and Persistence

The default Thing ID is `building:floor1:elevator`, mapped to MQTT-safe ID `building-floor1-elevator`.



Fig. 3. Reduced-scale four-floor, single-cabin ESP32-S3 elevator prototype (annotated). Floor position is tracked functionally by calibrated step counting; temperature, vibration, and load are simulated through ADC potentiometers. KY-024 and SPDT switches are design provisions for future closed-loop floor/door confirmation.

TABLE IV
REDUCED-SCALE ELEVATOR PROTOTYPE CHARACTERISTICS

Item	Value or evidence boundary
Physical configuration	One cabin, four logical floors numbered 0–3.
Frame dimensions	Approximately 84 cm high, 44 cm wide, 38 cm deep.
Cabin dimensions	15 cm cube, approximately 3375 cm ³ .
Movement timing	Approximately 6 s per adjacent floor, author-recorded prototype timing, not instrumented video/log evidence.
Controller	ESP32-S3 microcontroller running the custom elevator firmware.
Cabin motion	NEMA17 stepper through microstep driver; floor position tracked functionally by a calibrated step-count profile. KY-024 Hall sensors are a design provision for future closed-loop confirmation.
Door motion	DC motor through L298N/HW-095 H-bridge; timed travel profile. SPDT limit switches are a design provision for future closed-loop door confirmation.
Local HMI	16x4 I2C LCD, push buttons, emergency input, buzzer, fan relay.
Security input	MFR522 RFID support in firmware; physical UID scan evidence is outside current validation scope.
Safety telemetry	Temperature, vibration, and load currently simulated through ADC potentiometers.

Attributes store metadata and fleet-level status, while features represent subsystems such as `cabin`, `door`, `motor`, `security`, `fan`, `microcontroller`, `control`, `energy`, `performance`, `ai_analysis`, `predicted_failures`, and `maintenance_schedule` (Fig. 4). The `control` fea-

Thing building:floor1:elevator

Attributes (slowly varying metadata): fleet status, system_mode, risk_score

Features (subsystem state): cabin, door, motor, security, fan, microcontroller, energy, performance, ai_analysis, predicted_failures, maintenance_schedule

Feature control — separates intent from applied state:
intent: pending_command, dispatch_policy
device-applied: device_applied_policy

Fig. 4. Eclipse Ditto Thing model. Attributes hold slowly varying metadata; features hold subsystem state. The `control` feature explicitly separates command *intent* from *device-applied* state—the property that lets a rejected command leave the twin unchanged.

ture separates command intent from device-applied state. For dispatch, `control.dispatch_policy` is the authoritative intent selected by the engine, while `control.device_applied_policy` is reported by the firmware or simulator.

PostgreSQL/TimescaleDB stores raw telemetry, audit events, command decisions, notifications, maintenance work orders, system health records, and dispatch decisions. The final evidence export records seven application tables and 19,901 rows in `telemetry_raw` as of 2026-06-13 02:07:12.

The use of both Ditto and PostgreSQL is intentional. Ditto answers “what is the asset state now?” and PostgreSQL answers “what happened, why, when, and who requested it?” A rejected command is therefore as significant as an accepted command: it records that the system constrained an unsafe or unauthorized request and preserved the reason for later audit. This separation also supports future retention policies, fleet analytics, and incident reconstruction without overloading the twin with historical arrays.

E. Safety Gate

The command safety gate is implemented in the shared JavaScript package and mirrored in the `n8n` control workflow. It validates command allow-list, required fields, source, source agent, reason, confirmation, system mode, emergency stop, door state, overload, forced entry, risk threshold, target floor bounds, twin freshness, duplicate cooldown, dispatch-policy validity, and Ditto availability. If any mandatory predicate fails, the decision is persisted as rejected, `ditto_write_status` is `SKIPPED`, and the planned Ditto write list is empty.

$$\begin{aligned}
 \text{accept}(c, T) = & \text{allowlisted}(c) \wedge \text{valid}(c) \\
 & \wedge \text{authorized}(c) \wedge \text{fresh}(T) \\
 & \wedge \text{safeMode}(T) \wedge \text{safeInterlocks}(T) \quad (1) \\
 & \wedge \text{riskWithinLimit}(c, T) \\
 & \wedge \text{dittoReachable}(T).
 \end{aligned}$$

Here c is the requested command and T the current twin snapshot. The predicates correspond directly to the checks above: allowlisted and valid test the command catalog and required fields; authorized tests source, source agent, reason, and human confirmation; fresh requires a recent twin; safeMode excludes unsafe system modes; safeInterlocks tests emergency stop, door state, overload, and forced entry; riskWithinLimit bounds the risk score and target-floor range; and dittoReachable requires the twin store to be writable. A command is admitted only if all predicates hold; the implementation records all failing predicates rather than returning only the first failure.

Table V lists the principal rule families. The final software evidence confirms that rejected decisions carry no Ditto writes. In the live command-path runs, this property was not only unit-tested but also observed in database rows: every rejected command had `ditto_write_status = SKIPPED`.

The rule families in Table V operationalize Eq. (1) and span three kinds of protection: syntactic validity (catalog, fields, bounds), plant-state safety (mode, interlocks, freshness), and accountability (source, reason, confirmation, cooldown). The zero-write invariant in the final row is the architectural anchor: it turns every other family's rejection into a guaranteed non-event for the twin. Section V shows that evaluating the full predicate costs single-digit microseconds, so this breadth of checking does not translate into a latency argument against it.

F. Adaptive Dispatch Engine

The dispatch policy engine chooses the operating policy, not direct movement commands. Policies include `SCAN_COLLECTIVE`, `UP_PEAK`, `DOWN_PEAK`, `ECO_ENERGY`, `NEAREST_GREEDY`, `HEALTH_LIMP`, `BALANCED_INTERFLOOR`, and `SECURITY_RESTRICTED`. Hard safety overrides such as `EMERGENCY_STOP`, `OVERLOAD_HOLD`, `FULL_LOCKDOWN`, and `FIRE_RECALL` pre-empt optimization.

Brain A is the active deterministic scorer. Brain B is a softmax linear challenger over the shared feature vector. Brain B is logged in shadow mode and cannot become authoritative unless future evidence satisfies promotion gates and a human approves. This separation preserves explainability, rollback, and safety authority.

Crucially, these policies select dispatch behavior, not immediate cabin actuation: a policy command still passes the same safety gate and is then reported separately as intent (`control.dispatch_policy`) and applied state (`control.device_applied_policy`). The dispatch engine therefore adds an optimization layer without acquiring any authority the gate does not already mediate.

G. SCADA Dashboard

The Next.js dashboard reads Ditto through server-side APIs and history routes (Fig. 5). Dynamic values are handled in client hooks to avoid SSR hydration errors. The dashboard command center posts to `/api/commands`;

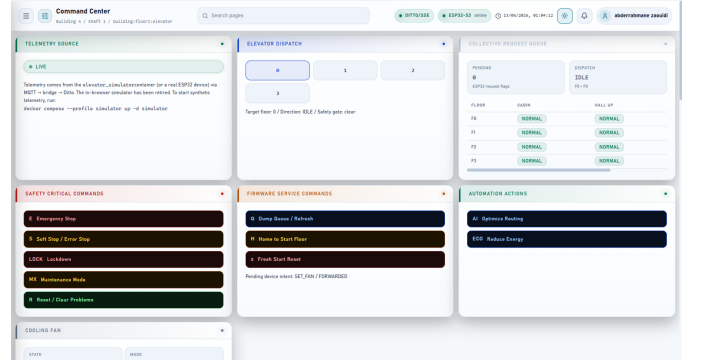


Fig. 5. SCADA command center. Operator commands post to `/api/commands` and pass the deterministic safety gate; recent command decisions and their accept/reject reasons are shown.

it does not publish MQTT commands. This preserves separation between UI, logic, state, and device transport.

The dashboard is therefore not a thin MQTT client. Its pages surface the live twin, historical telemetry, risk analysis, maintenance state, security state, command decisions, and dispatch policy. The server-side routes hide Ditto and database credentials from the browser and make the command path uniform for operator and workflow-originated requests. The current authentication boundary is prototype-level; production hardening requires server-side sessions, OIDC or equivalent identity management, role-based access control, HTTPS, and audit-linked user identities.

V. VALIDATION AND EVALUATION

A. Evidence Policy

Validation uses an evidence boundary rather than a blanket PASS claim. A result is marked PASS only when a repository artifact, exported log, screenshot, query output, or command transcript exists. Consistent with the framing in Section I, we report at three levels and never let evidence at one level imply another. *Software validation* covers logic and integration proven by automated tests, exports, and live-stack traces. *Integration validation* covers end-to-end behavior of the assembled stack observed on live infrastructure. *Documented hardware integration* records the built prototype, wiring, dimensions, and observed behavior, but it is not equivalent to certified or instrumented physical validation. The present paper contributes primarily at the first two levels; the third is documentation, and instrumented physical measurement is future work.

B. Validation Summary

Table VI summarizes the validated artifacts. The final live software run was executed on 2026-06-13 against the full Docker stack, the dashboard, Ditto, TimescaleDB, n8n, bridge, Mosquitto with authentication and ACLs, and an authenticated ESP32-compatible simulator.

The campaign intentionally uses qualified statuses. “PASS” is reserved for procedures with captured evidence and expected behavior. “Software PASS” describes logic

TABLE V
DETERMINISTIC COMMAND SAFETY-GATE RULE FAMILIES

Rule family	Examples	Safety effect
Command catalog	Unknown commands; unsupported aliases; missing required fields.	Prevents arbitrary payloads from becoming Ditto writes or device commands.
Source and accountability	Source must be authorized; source agent must be explicit; operator reason may be required.	Preserves traceability and blocks anonymous automation paths.
Human confirmation	Emergency stop, reset, lockdown, release, maintenance-mode transitions.	Prevents accidental state-changing actions and forces explicit operator intent.
Twin freshness and availability	Stale twin; Ditto unavailable; missing safety snapshot.	Avoids acting on old or incomplete operational state.
Safety interlocks	Emergency stop, overload, door open, forced entry, critical risk, unsafe system mode.	Blocks movement and unsafe recovery commands before they reach the device.
Bounds and policy validity	Target floor outside 0–3; unknown dispatch policy; invalid policy parameters.	Preserves physical and logical limits of the prototype.
Duplicate/cooldown	Same command, same target, same source within cooldown window.	Reduces repeated operator clicks or workflow churn.
Zero-write invariant	Any rejection clears the planned write array.	Ensures rejected commands cannot partially mutate Ditto state.

and integration proven by automated tests, logs, exports, or live-stack traces. “Documented integration” describes physical build evidence without a separate measurement campaign. “Outside current scope” marks work that must not be implied by simulator or configuration evidence. This vocabulary keeps the evaluation honest while still showing that the software architecture is reproducible.

The takeaway from Table VI is not any single row but the shape of the evidence: the software and integration layers of the architecture—transport security, bridge/twin synchronization, deterministic admission, dispatch, persistence, and the end-to-end command path—are each backed by a captured artifact, whereas the physical layer is deliberately marked as documented integration. The architecture is therefore reproducible and its command discipline is exercised on live infrastructure, while no claim is made that the physical mechanisms are instrumentally certified.

C. Command-Path Results

Two live command-path executions were captured. The first, part of the 2026-06-13 validation package, records nine decisions: an **EMERGENCY_STOP** without confirmation was rejected, a confirmed **EMERGENCY_STOP** was accepted and forwarded, a **MOVE_TO_FLOOR** request during unsafe state was rejected, **RESET_EMERGENCY** was accepted, unknown dispatch policies were rejected, and valid **SET_DISPATCH_POLICY** commands were accepted and adopted by the simulator.

The second is a scripted 57-command campaign (2026-07-05) driven through `POST /api/commands` against the full stack—dashboard, gate, Ditto, bridge, Mosquitto with authentication and ACLs, TimescaleDB, and the authenticated simulator with anomaly injection disabled for reproducibility. The campaign mixed accepted and rejected classes across eight live-exercisable rule families of Table V: unknown commands, missing required fields, missing operator reason, missing human confirmation, mode guards

traversed through complete emergency-stop, lockdown, and maintenance cycles with recovery, floor bounds, payload and policy validity, and duplicate/cooldown. Of 57 commands, 28 were accepted and 29 rejected, and every decision matched the catalog-predicted outcome. All 28 accepted commands completed their Ditto writes; all 29 rejected commands carried `ditto_write_status = SKIPPED`, and an independent export of `control_command_log` confirms both counts with zero invariant violations (`evidence/command-campaign/`). An initial campaign execution, retained in the same evidence directory, additionally exercised a plant-state interlock live: four **MOVE_TO_FLOOR** requests issued while the simulator autonomously served passengers were rejected with `door is open`.

These campaigns provide integration-level functional coverage of the live accept and reject branches per rule family; they are not long-duration robustness or fault-injection experiments. Families that require an unhealthy plant or infrastructure faults—twin staleness, Ditto unavailability, overload and forced-entry interlocks, risk-threshold caps—remain covered by the 33/33 gate assertions, and the 20,000-samples-per-class microbenchmarks below quantify decision cost. Fault-injection and long-duration soak campaigns remain future work.

D. Quantitative Evidence

Table VII separates measured software results, configured design parameters, and metrics deferred to future work. The distinction matters because configured intervals and the deterministic decision-time microbenchmarks reported below are useful evidence but are not substitutes for instrumented end-to-end latency, which remains future work.

The deterministic decision paths were microbenchmarked in-process over 20,000 samples per class on a single host, with per-call timing from `process.hrtime`; the raw distribution is archived in `evidence/perf/decision-latency.json`. The safety

TABLE VI
VALIDATION EVIDENCE SUMMARY

Subsystem	Status	Evidence	Boundary
MQTT topic convention	PASS	<code>evidence/mqtt/topic_validator_output.txt</code> : scanned 266 files, no unexpected legacy topic patterns, expected canonical topics present.	Software/configuration evidence.
MQTT authentication and ACLs	PASS	<code>acl_security_tests.txt</code> : anonymous and wrong-password connections denied; authenticated bridge subscriber received telemetry.	Broker-side evidence.
TLS publish/subscribe	PASS	Workspace <code>evidence/mqtt/05-roundtrip-tls.txt</code> : TLS publish exit 0 and subscriber receive exit 0 on 8883.	Field RTT not timed; loop-back RTT in Table VII.
Bridge and Ditto sync	PASS	<code>bridge_logs.txt</code> and Ditto Thing export show twin merges, online status, command-intent forwarding.	Software-stack evidence.
Command safety gate	PASS	<code>command_safety_gate_test_output.txt</code> : 33/33 assertions; live command rows show rejected commands with SKIPPED writes.	Deterministic software gate.
Dispatch engine	PASS	61/61 dispatch assertions across policy, safety, orchestrator, Brain B, and evaluation suites.	Brain B remains shadow-only.
Simulator regression	PASS	<code>simulator_pytest_output.txt</code> : 33/33 pytest items in 0.31 s.	Simulator software evidence, not physical hardware proof.
Database/history	PASS	Seven application tables; <code>telemetry_raw</code> count 19,901 rows on 2026-06-13.	Live database export.
End-to-end command path	PASS	Nine-decision run (2026-06-13) plus 57-command campaign (2026-07-05): 28 accepted, 29 rejected, all 29 rejections SKIPPED, database-verified (<code>evidence/command-campaign/</code>).	Integration-level functional coverage, not fault injection or soak testing; device endpoint was authenticated simulator.
Physical prototype	Documented integration	Thesis photographs, dimensions, wiring, and author-recorded adjacent-floor timing of approximately 6 s.	Not certified; not fully instrumented.

gate evaluates an accepted command in a median of $5.7 \mu\text{s}$ and a rejected command in $4.4\text{--}5.6 \mu\text{s}$, with the 99th percentile below $20 \mu\text{s}$. The active Brain A dispatch decision takes a median of $15.6 \mu\text{s}$ from the twin snapshot, and the full orchestrator loop—context construction, decision, gate preview, and command assembly—completes in a median of $22.4 \mu\text{s}$. These are CPU times of the decision logic, distinct from the test-suite durations reported by the runner, and they must not be read as end-to-end command latency, which additionally crosses the network, Ditto, and the device and remains future work. The deterministic admission adds about $5.3 \mu\text{s}$ of computation per command relative to a direct dashboard-to-MQTT serialization. This result matters for adoption: runtime-assurance and shielding layers are sometimes resisted on the grounds that inserting a filter between controller and plant costs time [24], [26], yet here the admission predicate is three orders of magnitude below the millisecond-scale MQTT round-trip and roughly six orders of magnitude below the multi-second physical floor-to-floor move. The safety filter is therefore not on the system's critical path, so the cost of the discipline is architectural rather than a measurable performance penalty at the level of command logic. Transport latency was measured separately over a loopback broker (`evidence/perf/mqtt-rtt.txt`): the steady-state MQTT round-trip is a median of 1.11 ms on the plain listener and 1.19 ms over the TLS listener (QoS 1, 500 samples each), which indicates that the encrypted hop adds negligible per-message overhead once the handshake is established. This supports a deployment conclusion: the always-on TLS baseline of Section IV-B

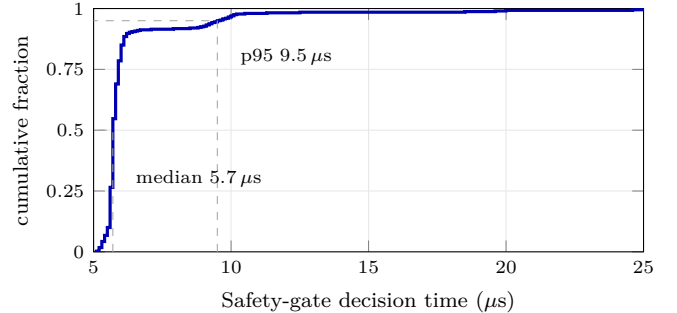


Fig. 6. Empirical CDF of the deterministic safety-gate decision time over 20,000 accepted-command evaluations (x clipped at $25 \mu\text{s}$; p99 is $19.6 \mu\text{s}$). The gate decides in single-digit microseconds; routing every command through it therefore adds negligible per-command overhead.

adds under 0.1 ms to the median round-trip, so transport security is not in tension with the telemetry duty cycle on this class of system.

The functional suites complement these microbenchmarks with semantic coverage. The 33 gate assertions pin the accept/reject behavior of every rule family in Table V, the 61 dispatch assertions pin policy selection, hard safety overrides, and the shadow-only status of Brain B, and the 33 simulator assertions pin the device-side MQTT contract. Their significance is architectural as much as evidential: the admission predicate is small and deterministic enough to be exhaustively unit-tested, which is precisely the property that runtime-assurance designs require of the high-assurance module [24].

TABLE VII
QUANTITATIVE EVIDENCE INVENTORY: MEASURED, CONFIGURED, AND FUTURE-WORK METRICS

Metric	Current value and interpretation	Classification
Adjacent-floor travel time	Approximately 6 s per floor; author-recorded prototype timing from the running reduced-scale rig, not instrumented.	Documented (author-recorded); instrumented timing is future work.
Telemetry publish interval	3000 ms when not moving (firmware constant; simulator tick log 3.00 s).	Configured.
Status heartbeat	5000 ms; bridge offline threshold 15000 ms.	Configured.
Bridge write interval	1000 ms; command-intent poll interval 500 ms.	Configured.
Command safety-gate validation	33/33 assertions; rejected commands carry zero Ditto writes. Suite duration is test-runner time, not latency.	Measured (functional).
Dispatch validation	61/61 assertions across policy, safety, orchestrator, Brain B, and evaluation suites.	Measured (functional).
Simulator regression	33/33 pytest items.	Measured (functional).
MQTT round-trip time	Loopback steady-state (QoS 1, 500 samples): local broker median 1.11 ms, TLS listener median 1.19 ms; plus functional TLS publish/subscribe (exit 0) on port 8883.	Measured.
Safety-gate decision time	Median 5.7 μ s for an accepted command, 4.4–5.6 μ s for rejected classes; p99 below 20 μ s (20,000 samples/class).	Measured (in-process).
Live dispatch decision time	Active Brain A: median 9.5 μ s (scorer) and 15.6 μ s including context build; full orchestrator loop 22.4 μ s.	Measured (in-process).
Cost of safety	Deterministic admission adds a median 5.3 μ s of in-process computation over a direct dashboard→MQTT serialization (0.42 μ s).	Measured (in-process).
End-to-end command latency	Coarse transcript indicates sub-second for one dispatch restore; no paired timestamps.	Future work.
Ditto synchronization delay	Bridge logs show repeated twin merges and online status; no paired timestamps.	Future work.
Reconnection time	Reconnect logic present; no timed fault-injection capture.	Future work.
ESP32 resource usage	No heap, CPU, or loop-timing capture (requires the physical board).	Future work (hardware).

E. Security Results

The broker evidence demonstrates deny-by-default authentication behavior: anonymous and wrong-password clients were rejected, while an authenticated bridge identity received telemetry. The ACL model also separates device and dashboard identities from command publication. This is stronger than a demo-only anonymous MQTT broker, but it remains a laboratory baseline. Production deployment would require credential rotation, certificate lifecycle management, network segmentation, service-to-service TLS, HTTPS for the dashboard and Ditto, secret management, and role-separated Ditto policies.

The threats addressed by the prototype, their implemented mitigations, and the remaining hardening steps are consolidated with the broader system limitations in Table VIII (Section VII). That table is an engineering boundary for a local research deployment, not a complete industrial security certification.

VI. DISCUSSION

A. Why Architectural Discipline Matters

The platform's main contribution is not a faster dispatch benchmark. It is a command and state discipline for cyber-physical automation. Ditto gives every consumer the same representation of the elevator. MQTT remains a transport boundary. The dashboard is supervisory, not authoritative over devices. Workflows may recommend or request, but a deterministic gate decides. The database records both accepted and rejected actions, which makes post-event audit possible. The microbenchmarks indicate that this discipline can be adopted without a measurable

performance penalty at the level of command logic: the gate decides in a median of 5.7 μ s and adds about 5.3 μ s of computation per command over an unsafe direct-publish path, so the cost of routing every command through deterministic admission is architectural rather than a performance trade-off.

The reject-with-zero-write invariant deserves emphasis as a safety property rather than an implementation detail. It proves that an unsafe or unauthorized request is recorded for audit yet cannot partially mutate the twin: the state authority is never advanced by a command the gate did not admit. This matters because a state authority that could be nudged by rejected intent would let an untrusted client influence downstream consumers—dispatch, workflows, and the dashboard all read Ditto—even without a full command. The property is the concrete, elevator-specific realization of the runtime-assurance principle that an untrusted component must not move the system out of its admissible set [24], [26], [27], and it is what makes gap dimension (ii), auditable command authority, more than a logging convention.

This discipline is particularly important for LLM-augmented systems. The optional LLM can produce a summary, but it cannot change the command predicate, bypass interlocks, or write direct device commands. Likewise, the ML dispatch challenger can be evaluated against Brain A without becoming active. The result is bounded automation rather than autonomous elevator control.

B. Strengths

The implementation integrates a real embedded prototype, a simulator with the same MQTT contract, broker

authentication and ACLs, TLS on the ESP32-side hop, a Ditto Thing model, bridge normalization, duplicate write suppression, database persistence, deterministic command admission, n8n workflows, a dispatch engine, and a SCADA dashboard. The evidence package is explicit about what is validated and what is only documented, and the deterministic decision paths are now quantified. Instrumented end-to-end latency and embedded-resource profiling remain the main measurement extensions.

C. Scope Boundaries

The present platform is not a certified elevator controller. Floor position is tracked functionally, in open loop, by a calibrated step-count profile; the KY-024 floor sensors and SPDT door limit switches are design provisions for future closed-loop confirmation, which would add independent, redundant position and door-state evidence rather than a currently missing basic function. Temperature, vibration, and load are simulated through ADC potentiometers. Physical RFID validation requires UID captures and accepted/denied scan evidence. The final software end-to-end run used an authenticated simulator as the device endpoint. These boundaries do not invalidate the architecture, but they must remain visible in the paper.

VII. LIMITATIONS AND FUTURE WORK

Table VIII consolidates, in a single view, the security threats addressed by the prototype (Section V) and the broader system limitations, each paired with its current mitigation or status and the corresponding hardening step. The most important limitation is certification scope: the system is a laboratory prototype and must not be presented as a deployable building elevator controller. The software architecture can inform safer connected-elevator supervision, but industrial adoption would require certified controllers, independent safety circuits, formal hazard analysis, and compliance review.

Read as a whole, the table separates two kinds of future work that are often conflated. The security and governance rows are *deployment* hardening: the architectural discipline is in place, but production identity, secret, and certificate management must be added before field use. The sensing, latency, and resource rows are *measurement and instrumentation* work: they convert a software-validated architecture into a quantitatively characterized system without changing its command-authority design.

VIII. CONCLUSION

This paper condensed a master's thesis platform into a journal-style architecture contribution for trustworthy elevator digital twins. It advanced three claims, matching the three contributions stated in Section I. First, the platform is a reproducible, self-hosted supervision architecture connecting an ESP32-S3 elevator prototype and simulator to Mosquitto, a Node.js bridge, Eclipse Ditto, PostgreSQL/TimescaleDB, bounded n8n workflows, an adaptive dispatch engine, and a Next.js SCADA dashboard.

Second, its central safety property is a command-authority discipline that is architectural rather than incidental: commands are never direct UI-to-device actions, but are audited, safety-gated, written as Ditto intent only when accepted, forwarded by the bridge, and constrained again by device-side interlocks, so a rejected command mutates no state. Third, this discipline was quantified and shown to be inexpensive.

The evidence supports these claims at the software and integration levels rather than as certified physical validation. A total of 127 automated assertions passed, MQTT authentication and ACLs were exercised, TLS publish/subscribe was demonstrated, and Ditto and database exports were captured. On the live stack, a nine-decision run and a 57-command campaign across eight gate rule families recorded 28 accepted and 29 rejected commands, with every rejection carrying zero Ditto writes—database-verified, integration-level functional coverage of the command path. The deterministic decision paths were also quantified: the safety gate admits a command in a median of 5.7 μ s and adds about 5.3 μ s over a direct publish, so the discipline imposes negligible computational cost on command logic. Floor position is tracked functionally by calibrated step counting; closed-loop sensor confirmation, fault-injection and soak campaigns, instrumented end-to-end latency, and embedded-resource profiling remain future work. Within these boundaries, the platform is offered as a safety-first, auditable, bounded-autonomy architecture for smart-elevator digital twins.

DATA AND ARTIFACT AVAILABILITY

The implementation and the thesis evidence package dated 2026-06-13 are maintained in the project repository at <https://github.com/abdelrahmen-zaouidi/smart-elevator-digital-twin> (Apache License 2.0). Secrets, passwords, and private environment files are excluded. A camera-ready submission provides a sanitized artifact bundle or repository snapshot with the evidence paths listed in this paper.

AUTHOR CONTRIBUTIONS (CREDIT)

Abderrahmane Zaouidi: conceptualization of the software architecture, methodology, software, system integration, validation, investigation, visualization, data curation, and original draft preparation. Mohamed Hachem Bengherbia: implementation support, technical review, validation support, and manuscript review. Mounir Bouhedda: original research idea, supervision, project administration, methodology guidance, and manuscript review and editing.

ACKNOWLEDGMENT

The authors acknowledge LSEA, University of Médéa, for the academic supervision and laboratory context of this work.

TABLE VIII
CONSOLIDATED LIMITATIONS AND THREATS MAPPED TO CURRENT MITIGATION AND FUTURE HARDENING

Limitation or threat	Implemented mitigation or current status	Required hardening step
Laboratory prototype, not certified equipment; firmware interlocks are not safety-rated circuits. Floor position is open-loop (calibrated step counting); no independent closed-loop confirmation. Door end-position confirmation not integrated (SPDT mounted as a provision). Temperature, vibration, and load are simulated (ADC potentiometers). RFID physical access-control evidence incomplete. Unauthorized MQTT publish.	Firmware-side interlocks check emergency stop, overload, door state, and floor bounds; the deterministic gate blocks unsafe backend commands. Calibrated step-count profile drives repeatable floor-to-floor motion in the prototype. Timed door-travel profile. Feature envelope and thresholds exercised with simulated proxies.	Certified safety hardware, independent protection circuits, formal hazard analysis, and jurisdiction-specific elevator-code compliance. Integrate KY-024 pins, debouncing, and homing for redundant, timestamped floor-by-floor confirmation.
Browser-to-device command bypass; prototype-level dashboard authentication.	Dashboard has no MQTT command-publish authority; commands use the server-side API, safety gate, audit, Ditto intent, and bridge forwarding.	Server-side identity, RBAC, HTTPS, audit-linked user IDs, and secure cookies.
Credential leakage in workflows. Replay or duplicate command.	Secrets excluded from exported workflows and environment templates. Bridge tracks forwarded command IDs and filters stale intents; gate applies cooldown.	Restricted n8n access, secret manager, and audit-linked credentials. Shared cooldown/command memory for multi-replica deployments.
Unsafe workflow or LLM recommendation.	Workflows may request commands, but the deterministic gate decides; optional LLM output is explanation only.	Prompt/output logging and model governance if LLM explanations are enabled.
Brain B dispatch challenger is shadow-only. Stale or inconsistent operational state.	Logged in shadow mode; cannot become authoritative. Gate checks twin freshness; bridge writes Ditto as source of truth; database is historical.	Keep shadow-only until real outcome data, promotion-gate success, a rollback plan, and human approval exist. Instrumented clock synchronization and freshness-delay measurements.
Instrumented latency and embedded-resource metrics missing. Live command evidence covers the healthy-plant configuration.	Configured intervals and in-process decision-time microbenchmarks captured. 57-command campaign across eight rule families: 28 accepted, 29 rejected, zero-write invariant on 29/29 rejections (database-verified); a prior execution also exercised the door-open interlock live.	Synchronized end-to-end command, MQTT RTT, Ditto-sync, dispatch, and reconnection timing; ESP32 heap, loop-timing, and TLS-memory logs. Fault-injection campaigns (broker or twin outage, stale twin, overload) and long-duration soak runs.
Audit tables are ordinary database rows.	Command decisions, audit events, and dispatch records persisted.	Retention policy, backups, and tamper-evident or append-only audit export.

REFERENCES

- [1] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*. Springer, 2017, pp. 85–113.
- [2] E. H. Glaessgen and D. S. Stargel, "The digital twin paradigm for future NASA and U.S. Air Force vehicles," in *53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference*, 2012.
- [3] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.
- [4] Q. Qi, F. Tao, T. Hu, N. Anwer, A. Liu, Y. Wei, L. Wang, and A. Y. C. Nee, "Enabling technologies and tools for digital twin," *Journal of Manufacturing Systems*, vol. 58, pp. 3–21, 2021.
- [5] W. Kritzing, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [6] R. Minerva, G. M. Lee, and N. Crespi, "Digital twin in the IoT context: A survey on technical features, scenarios, and architectural models," *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1785–1824, 2020.
- [7] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital twin: Enabling technologies, challenges and open research," *IEEE Access*, vol. 8, pp. 108 952–108 971, 2020.
- [8] Eclipse Ditto Project, "Eclipse Ditto documentation," <https://eclipse.dev/ditto/>, 2026.
- [9] M. Lagally, R. Matsukura, M. McCool, and K. Toumura, "Web of Things (WoT) architecture 1.1," <https://www.w3.org/TR/wot-architecture11/>, 2023.
- [10] R. van Dinter, B. Tekinerdogan, and C. Catal, "Predictive maintenance using digital twins: A systematic literature review," *Information and Software Technology*, vol. 151, p. 107008, 2022.
- [11] T. Zonta, C. A. da Costa, R. da Rosa Righi, M. J. de Lima, E. S. da Trindade, and G. P. Li, "Predictive maintenance in the Industry 4.0: A systematic literature review," *Computers & Industrial Engineering*, vol. 150, p. 106889, 2020.
- [12] U. Hunkeler, H. L. Truong, and A. Stanford-Clark, "MQTT-S—a publish/subscribe protocol for wireless sensor networks," in *3rd International Conference on Communication Systems Software and Middleware*, 2008, pp. 791–798.
- [13] OASIS, "MQTT version 5.0," <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019.
- [14] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP," in *2017 IEEE International Systems Engineering Symposium*, 2017, pp. 1–7.

- [15] R. A. Light, “Mosquitto: Server and client implementation of the MQTT protocol,” *Journal of Open Source Software*, vol. 2, no. 13, p. 265, 2017.
- [16] K. Stouffer, S. Lightman, V. Pillitteri, M. Abrams, and A. Hahn, “Guide to industrial control systems (ICS) security,” National Institute of Standards and Technology, Tech. Rep. SP 800-82 Rev. 2, 2015.
- [17] IEC, “IEC 62443-3-3:2013: Industrial communication networks – network and system security – part 3-3: System security requirements and security levels,” <https://webstore.iec.ch/en/publication/7033>, 2013.
- [18] National Institute of Standards and Technology, “The NIST cybersecurity framework (CSF) 2.0,” National Institute of Standards and Technology, Tech. Rep. NIST CSWP 29, 2024.
- [19] V. Hassija, V. Chamola, V. Saxena, D. Jain, P. Goyal, and B. Sikdar, “A survey on IoT security: Application areas, security threats, and solution architectures,” *IEEE Access*, vol. 7, pp. 82 721–82 743, 2019.
- [20] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.
- [21] A. Dorri, S. S. Kanhere, and R. Jurdak, “Multi-agent systems: A survey,” *IEEE Access*, vol. 6, pp. 28 573–28 593, 2018.
- [22] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding *et al.*, “The rise and potential of large language model based agents: A survey,” *arXiv preprint arXiv:2309.07864*, 2023.
- [23] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [24] L. Sha, “Using simplicity to control complexity,” *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.
- [25] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.
- [26] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI-18)*, 2018, pp. 2669–2678.
- [27] P. J. G. Ramadge and W. M. Wonham, “The control of discrete event systems,” *Proceedings of the IEEE*, vol. 77, no. 1, pp. 81–98, 1989.
- [28] Espressif Systems, “ESP32-S3 series datasheet,” https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf, 2024.
- [29] —, “ESP-MQTT: ESP32-S3 ESP-IDF programming guide,” <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/protocols/mqtt.html>, 2026.